

DESIGN PATTERNS

Recognition-first GoF cheatsheet

All 23 creational, structural, and behavioral patterns - concise theory, selection hacks, trade-offs, memorable vector analogies, and compact Java implementation shapes.

23 GoF patterns

recognition heuristics

trade-offs

Java skeletons



CREATIONAL

5 PATTERNS

How objects are born



STRUCTURAL

7 PATTERNS

How objects are connected



BEHAVIORAL

11 PATTERNS

How objects collaborate

Do not memorize names. Diagnose forces.

A pattern is a reusable trade-off for a recurring pressure - not a badge to add to every design.

1

Locate the pressure

What is painful now: object creation, coupling, conditionals, traversal, memory, or coordination?

2

Find the axis of change

Which decision is likely to vary independently: product type, platform, algorithm, lifecycle, or operation?

3

Choose binding time

Must the choice be fixed in code, configured at startup, selected per request, or changed during object life?

4

Pay the cost consciously

Name the extra indirection/classes and the simpler option you rejected. Patterns optimize change, not line count.

THE FOUR COORDINATES OF A PATTERN DECISION

QUESTION	TYPICAL ANSWERS	WHAT IT REVEALS
What varies?	type / family / algorithm / state / operation	candidate pattern family
Who chooses?	caller / creator / context / event / framework	ownership of variation
When chosen?	compile / startup / runtime / per call / transition	binding mechanism
What stays stable?	client contract / workflow / tree / product family	the abstraction boundary
What must disappear?	new, switch, instanceof, peer web, duplication	the code smell to refactor

THE 60-SECOND INTERVIEW ANSWER

CONTEXT	Name the changing requirement.
FORCE	Explain the coupling/conditional/duplication.
CHOICE	State the pattern and its intent.
SHAPE	Name roles and delegation direction.
TRADE-OFF	Mention cost and when you would not use it.

RECOGNITION HACKS: TRANSLATE SMELLS INTO QUESTIONS

SCATTERED NEW

Who should own creation?

GIANT SWITCH

What varies independently?

SUBCLASS EXPLOSION

Can composition split axes/layers?

PEER DEPENDENCY WEB

Can one coordinator own interaction?

REPEATED TRAVERSAL

Can traversal become an object?

MEMORY DUPLICATION

What state can be shared immutably?

The complete GoF atlas

Read the verb first. The category and intent matter more than the class diagram.

CREATIONAL

5 PATTERNS

1

Singleton

Guarantee one controlled instance and one global access point.

2

Factory Method

Let a creator defer the concrete product choice to a hook or subtype.

3

Abstract Factory

Create a compatible family of products without naming concrete classes.

4

Builder

Construct a complex object through named, validated steps.

5

Prototype

Create new objects by copying a configured exemplar.

STRUCTURAL

7 PATTERNS

1

Adapter

Translate an existing interface into the interface a client expects.

2

Bridge

Separate an abstraction from its implementation so both axes can vary independently.

3

Composite

Treat individual objects and recursive groups through one uniform interface.

4

Decorator

Add stackable responsibilities to an object while preserving its interface.

5

Facade

Expose a small task-oriented entry point over a complex subsystem.

6

Flyweight

Share immutable intrinsic state across many lightweight contextual objects.

7

Proxy

Stand in for another object to control access, lifecycle, or location.

BEHAVIORAL

11 PATTERNS

1

Chain of Responsibility

Pass a request through an ordered chain until handlers process or forward it.

2

Command

Encapsulate a request as an object with everything needed to execute it.

3

Interpreter

Represent a small grammar as objects that evaluate expressions.

4

Iterator

Traverse a collection sequentially without exposing its representation.

5

Mediator

Centralize interaction rules so peer objects do not reference each other directly.

6

Memento

Capture and restore an object snapshot without exposing its internals.

7

Observer

Notify a dynamic set of subscribers when a subject changes or emits an event.

8

State

Let an object change behavior when its internal state changes.

9

Strategy

Encapsulate interchangeable algorithms behind one contract.

10

Template Method

Fix an algorithm skeleton in a base class while subclasses customize selected steps.

11

Visitor

Add operations across a stable object structure without modifying element classes.

Fast pattern selector

Start with the dominant force. Follow the branch only until one intent fits; then verify its trade-off.

CREATION

Exactly one identity?

Singleton

Copy an existing setup?

Prototype

Many optional construction inputs?

Builder

Compatible product family?

Abstract Factory

One product choice inside workflow?

Factory Method

STRUCTURE

Incompatible contract?

Adapter

Simple front door to subsystem?

Facade

Control access to same contract?

Proxy

Stack optional behavior?

Decorator

Part-whole recursive tree?

Composite

Two independent dimensions?

Bridge

Mass duplicate intrinsic state?

Flyweight

BEHAVIOR

Caller chooses algorithm?

Strategy

Lifecycle mode changes behavior?

State

Fixed workflow with custom steps?

Template Method

Dynamic one-to-many notification?

Observer

Central peer coordination?

Mediator

Request must be queued/undone?

Command

Ordered handling pipeline?

Chain of Responsibility

Restore private snapshot?

Memento

Hide traversal mechanics?

Iterator

New operations over stable types?

Visitor

Evaluate a tiny grammar?

Interpreter

CREATIONAL

Control how objects are born.

Which construction decision should the client stop owning?

CATEGORY RULE

Move volatile creation behind a stable abstraction - but keep object ownership and lifetime explicit.

SELECTION LENS

PATTERN	DECISION VERB	DECISIVE RECOGNITION HACK	BINDING SHAPE
Singleton	How many?	Exactly one identity is a correctness invariant	application lifetime
Factory Method	Which product?	One creation choice varies inside stable workflow	hook / subtype
Abstract Factory	Which family?	Related products must switch together	factory object
Builder	How configured?	Many named options + final validation	step-by-step
Prototype	Which exemplar?	Existing configured state is the recipe	copy at runtime

REMEMBER THE DOMINANT NOUN

CONSTRUCTION

Which construction decision should the client stop owning?

COMMON CATEGORY MISTAKES

- Hiding every constructor behind a factory
- Using Singleton as global convenience
- Adding Builder to tiny value objects

BEST INTERVIEW MOVE

Compare your choice with the closest look-alike and explain the intent difference.

Singleton

Guarantee one controlled instance and one global access point.

RECOGNITION RADAR

CORE SIGNAL The invariant is identity: the process must observe exactly one shared instance.

- Multiple instances would break correctness, not merely waste memory.
- Every caller needs the same coordinator, registry, clock, or process-wide resource.
- Construction must be hidden and thread-safe; lifetime is application-wide.

WHY THIS IS THE BEST FIT

- It centralizes ownership and enforces the one-instance invariant at construction time.
- A lazy holder gives simple, thread-safe initialization without explicit locking.

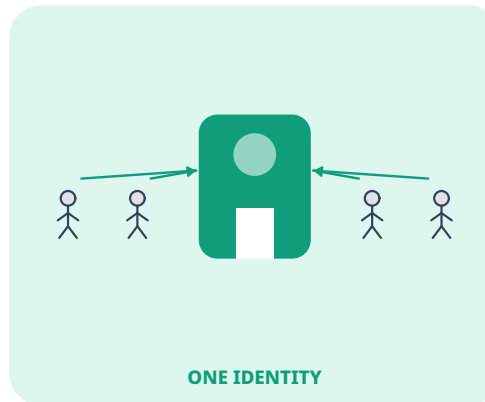
TRADE-OFFS AND GUARDRAILS

- ! COST** Global state, hidden dependencies, difficult isolation in tests, and lifecycle coupling.
- ! AVOID WHEN** When "one per app" is only convenience. Prefer dependency injection with singleton scope.

INTERVIEW-READY SENTENCE

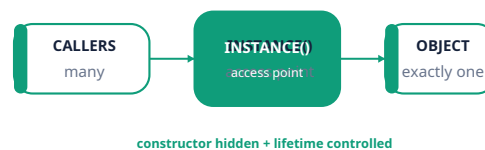
“ Use Singleton only when uniqueness is a domain invariant; otherwise inject one shared object.

MEMORY ANCHOR



One lighthouse, many ships: a single source coordinates everyone.

STRUCTURE SNAPSHOT



FINGERPRINT

One instance Early or lazy Private constructor

CONFUSION KILLER Singleton controls quantity. A static utility exposes functions but has no substitutable object identity.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: AppRegistry is the single shared object; instance() is the
2 // access point.
3 // Follow the lifecycle: private constructor -> lazy Holder -> one stored
4 // instance.
5
6 public final class AppRegistry {
7     // Only this class can call the constructor.
8     private AppRegistry() {}
9
10    // Holder loads only on the first instance() call.
11    private static class Holder {
12        private static final AppRegistry INSTANCE =
13            new AppRegistry();
14    }
15
16    // Every caller receives the same object reference.
17    public static AppRegistry instance() {
18        return Holder.INSTANCE;
19    }
20
21    public void register(String key, Object service) {
22        // Store shared application services here.
23    }
24 }
25
26 // The JVM class-initialization rule makes this thread-safe.
27 // Do not store request-specific data in a singleton.
28
29 // Client code: direct construction is impossible.
30 AppRegistry a = AppRegistry.instance();
31 AppRegistry b = AppRegistry.instance();
32 a.register("mail", mailService);
33 boolean same = (a == b); // true
34
35 // Prefer dependency injection unless identity must be unique.
36
37 // PATTERN CLUE: correctness requires one identity, not merely equal
38 // values.
```

PRODUCTION NOTES

- Prefer enum Singleton when serialization/reflection hardening matters.
- Keep mutable state minimal; synchronize state, not just construction.
- Do not pass instance() calls deep through business code - inject the dependency.

Factory Method

Let a creator defer the concrete product choice to a hook or subtype.

RECOGNITION RADAR

CORE SIGNAL Creation varies inside an otherwise stable operation.

- A workflow is stable, but one product instantiated inside it changes by configuration or subclass.
- Client code should depend on a Product interface, not scattered new ConcreteProduct calls.
- You need one overridable creation decision rather than a whole family of related objects.

WHY THIS IS THE BEST FIT

- It localizes construction variation while preserving the creator workflow.
- New product types can be added without changing the code that consumes the product interface.

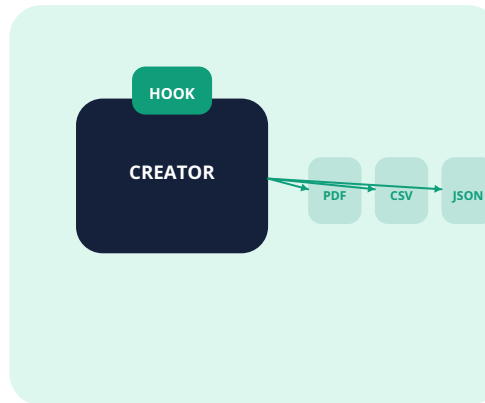
TRADE-OFFS AND GUARDRAILS

- ⚠ **COST**
Extra creator hierarchy or indirection; simple mappings may be clearer as a function or registry.
- ⚠ **AVOID WHEN**
When construction is trivial and closed; a constructor or parameter may be enough.

INTERVIEW-READY SENTENCE

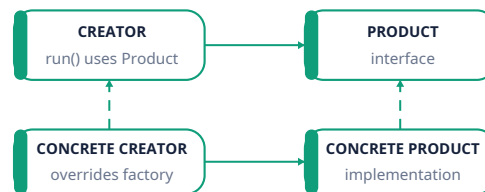
“ Choose Factory Method when one product choice varies inside a stable creator algorithm.

MEMORY ANCHOR



A machine keeps the same workflow while its production head is replaceable.

STRUCTURE SNAPSHOT



FINGERPRINT

One product axis Creation hook Creator uses product

CONFUSION KILLER Factory Method creates one polymorphic product. Abstract Factory creates a compatible product family.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: Parser is the product; ImportJob is the creator;
2 // createParser() is the factory method.
3 // run() owns the stable workflow; each subclass changes only object
4 // creation.
5
6 interface Parser {
7     Result parse(String input);
8 }
9
10 // Creator owns the stable workflow.
11 abstract class ImportJob {
12     // Subclasses decide which product is created.
13     protected abstract Parser createParser();
14
15     public final Result run(String input) {
16         Parser parser = createParser();
17         return parser.parse(input);
18     }
19 }
20
21 final class CsvJob extends ImportJob {
22     @Override
23     protected Parser createParser() {
24         return new CsvParser();
25     }
26 }
27
28 final class JsonJob extends ImportJob {
29     protected Parser createParser() {
30         return new JsonParser();
31     }
32 }
33
34 // Client selects a creator; run() remains unchanged.
35 ImportJob job = new CsvJob();
36 Result result = job.run(rawText);
37
38 // PATTERN CLUE: subclasses must choose a product without duplicating the
39 // workflow.
```

PRODUCTION NOTES

- The creator may be abstract, or a concrete class with a default factory method.
- Keep product use in the creator; returning objects only from a static helper is a Simple Factory.
- Registries of Supplier<Product> often replace subclass proliferation.

Abstract Factory

Create a compatible family of products without naming concrete classes.

RECOGNITION RADAR

CORE SIGNAL The family must switch together, and products within a family must remain compatible.

- Several related products vary as one theme: platform, vendor, database, region, or UI toolkit.
- Mixing concrete products from different families would be invalid or visually inconsistent.
- Client logic should remain unchanged when the entire family is swapped.

WHY THIS IS THE BEST FIT

- A single factory object makes family consistency explicit and hard to violate.
- The client sees only abstract product interfaces, so platform/vendor details stay at composition root.

TRADE-OFFS AND GUARDRAILS

! COST

Adding a new product kind changes every factory; the pattern favors new families over new product types.

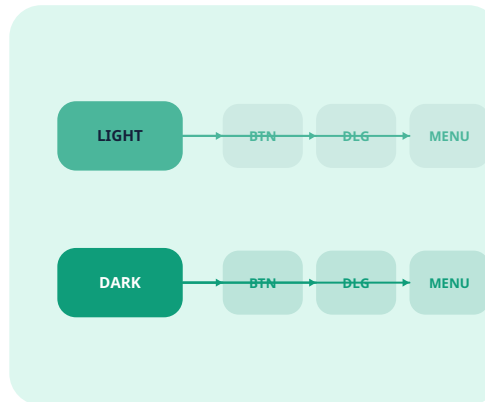
! AVOID WHEN

When products do not form a real compatibility family or only one object varies.

INTERVIEW-READY SENTENCE

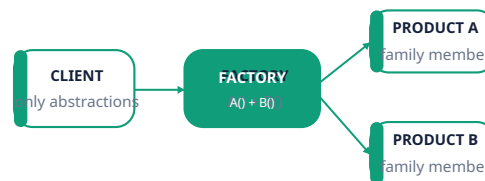
“ Use Abstract Factory when multiple related construction choices must change atomically as a family.

MEMORY ANCHOR



One style kit supplies matching chair, desk, and lamp - never a mismatched room.

STRUCTURE SNAPSHOT



FINGERPRINT

Product family Runtime family switch

CONFUSION KILLER

Builder assembles one complex object step by step. Abstract Factory returns multiple related products.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: UiFactory creates a compatible Button + Dialog product family.
2 // Screen uses only factory interfaces, so it cannot accidentally mix
3 // themes.
4
5 interface Button { void paint(); }
6 interface Dialog { void open(); }
7
8 // Creates every product in one compatible family.
9 interface UiFactory {
10     Button createButton();
11     Dialog createDialog();
12 }
13
14 final class DarkUiFactory implements UiFactory {
15     public Button createButton() {
16         return new DarkButton();
17     }
18     public Dialog createDialog() {
19         return new DarkDialog();
20     }
21 }
22
23 final class Screen {
24     private final Button button;
25     private final Dialog dialog;
26
27     Screen(UiFactory factory) {
28         // Both products come from the same theme.
29         button = factory.createButton();
30         dialog = factory.createDialog();
31     }
32
33     void show() { button.paint(); dialog.open(); }
34 }
35
36 // Replace one factory to replace the whole family.
37 Screen screen = new Screen(new DarkUiFactory());
38
39 // PATTERN CLUE: one configuration choice must swap several related
40 // products together.
```

PRODUCTION NOTES

- Inject one factory at the composition root; do not ask for family type throughout the domain.
- Factories are often stateless and can be lambdas only when they create one product.
- Use tests with a fake factory to build a coherent fake family.

Builder

Construct a complex object through named, validated steps.

RECOGNITION RADAR

CORE SIGNAL Construction has many optional parameters, ordering concerns, or cross-field invariants.

- Constructors are unreadable, telescoping, or full of null/default arguments.
- The object should be immutable after creation, but setup needs several named choices.
- Validation is best performed once after all construction inputs are known.

WHY THIS IS THE BEST FIT

- Named methods make call sites self-documenting and remove parameter-order mistakes.
- build() provides one boundary for defaults and whole-object invariant validation.

TRADE-OFFS AND GUARDRAILS

! COST

Boilerplate and possible duplication of fields; unnecessary for tiny stable value objects.

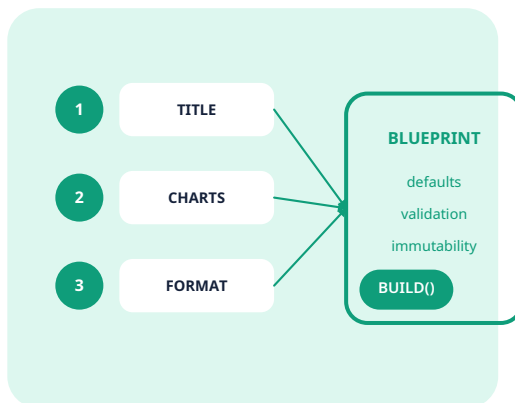
! AVOID WHEN

When the object has two or three mandatory fields and no meaningful optional configuration.

INTERVIEW-READY SENTENCE

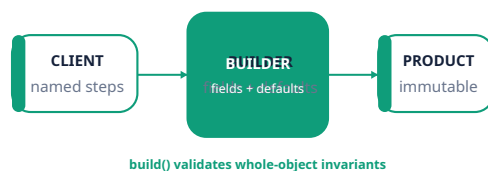
“ Use Builder when creation itself is a mini-workflow and the final object should be valid and immutable.

MEMORY ANCHOR



A blueprint collects decisions; build() opens the door only when the house is valid.

STRUCTURE SNAPSHOT



FINGERPRINT

Many options Build-time binding Fluent assembly

CONFUSION KILLER

Builder controls how one object is assembled. Factory controls which concrete object is selected.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: Report is the immutable product; Builder holds temporary
2 // construction state.
3 // Set optional values fluently, validate once, then create the finished
4 // object.
5
6 final class Report {
7     private final String title;
8     private final boolean charts;
9
10    private Report(Builder b) {
11        title = b.title;
12        charts = b.charts;
13    }
14
15    static final class Builder {
16        private String title;    // required value
17        private boolean charts; // optional: false
18
19        Builder title(String value) {
20            title = value;
21            return this; // enables fluent chaining
22        }
23
24        Builder charts(boolean value) {
25            charts = value;
26            return this;
27        }
28
29        Report build() {
30            if (title == null || title.isBlank())
31                throw new IllegalStateException("title required");
32            return new Report(this);
33        }
34    }
35
36    // Readable construction; no telescoping constructor.
37    Report report = new Report.Builder()
38        .title("Sales").charts(true).build();
39
40    // PATTERN CLUE: many optional parameters or ordered steps make
41    // constructors unreadable.
```

PRODUCTION NOTES

- Separate required constructor arguments from optional fluent setters.
- Make builders single-use or document reuse semantics; mutable builders are not thread-safe.
- A Director is optional; it is useful only for reusable construction recipes.

Prototype

Create new objects by copying a configured exemplar.

RECOGNITION RADAR

CORE SIGNAL An initialized object is the best specification for the next object.

- Creation is expensive because setup, parsing, calibration, or graph assembly has already happened.
- Runtime-defined templates must be cloned without knowing their concrete classes.
- New objects start almost identical to an existing object and then diverge slightly.

WHY THIS IS THE BEST FIT

- Copying preserves a complex configuration without re-running the entire construction process.
- A prototype registry lets users or configuration define available object templates at runtime.

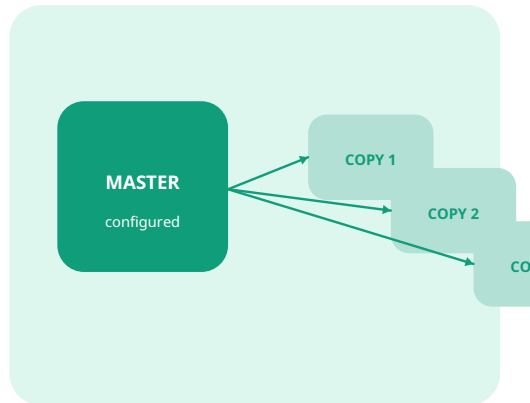
TRADE-OFFS AND GUARDRAILS

- ⚠ **COST**
Deep-vs-shallow copy bugs, identity/reference confusion, and difficult copying of resources.
- ⚠ **AVOID WHEN**
For objects holding sockets, threads, database sessions, or ownership that cannot be safely duplicated.

INTERVIEW-READY SENTENCE

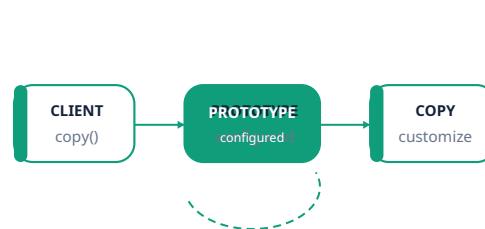
“ Choose Prototype when existing state is the creation recipe and copying is safer or cheaper than reconstruction.

MEMORY ANCHOR



A master stamp produces variations faster than rebuilding every detail.

STRUCTURE SNAPSHOT



FINGERPRINT

Copy exemplar Runtime template Clone then customize

CONFUSION KILLER Prototype copies an existing instance. Builder reconstructs from explicit construction inputs.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: an existing Campaign is the prototype; copy() creates a new
2 // starting point.
3 // Mutable nested data must be copied so the new object can change
4 // independently.
5
6 final class Campaign {
7     private String name;
8     private final List<String> channels;
9
10    Campaign(String name, List<String> channels) {
11        this.name = name;
12        this.channels = new ArrayList<>(channels);
13    }
14
15    // Copy constructor is the cloning mechanism.
16    private Campaign(Campaign source) {
17        name = source.name;
18
19        // Deep-copy mutable state; do not share the list.
20        channels = new ArrayList<>(source.channels);
21    }
22
23    Campaign copy() {
24        return new Campaign(this);
25    }
26
27    void rename(String value) { name = value; }
28    void addChannel(String value) { channels.add(value); }
29 }
30
31 // Begin with a preconfigured exemplar.
32 Campaign summer = new Campaign(
33     "Summer", List.of("email", "social"));
34
35 Campaign winter = summer.copy();
36 winter.rename("Winter");
37 winter.addChannel("sms"); // summer is unaffected
38
39 // PATTERN CLUE: cloning a configured exemplar is easier than rebuilding
40 // from zero.
```

PRODUCTION NOTES

- Prefer explicit copy constructors/factories over Java Cloneable.
- Document whether nested objects are deep-copied, shared, or immutable.
- Assign fresh identifiers after copying unless identity is intentionally preserved.

STRUCTURAL

Control how objects are connected.

What relationship is causing coupling, duplication, or a class explosion?

CATEGORY RULE

Preserve a stable client-facing contract while changing translation, layering, access, hierarchy, or sharing.

SELECTION LENS

PATTERN	DECISION VERB	DECISIVE RECOGNITION HACK	BINDING SHAPE
Adapter	Translate	Existing capability, incompatible interface	boundary wrapper
Bridge	Split axes	Two independent dimensions multiply subclasses	two hierarchies
Composite	Unify one/many	Recursive part-whole tree	uniform Component
Decorator	Layer behavior	Optional stackable responsibilities	same-interface wrapper
Facade	Simplify	Common workflow spans a complex subsystem	task API
Flyweight	Share state	Mass duplicate immutable intrinsic data	cache by value
Proxy	Control access	Same contract, guarded/remote/lazy subject	stand-in

REMEMBER THE DOMINANT NOUN

RELATIONSHIP

What relationship is causing coupling, duplication, or a class explosion?

COMMON CATEGORY MISTAKES

- Confusing Adapter with Facade
- Calling every wrapper Decorator
- Using Flyweight without profiling

BEST INTERVIEW MOVE

Compare your choice with the closest look-alike and explain the intent difference.

Adapter

Translate an existing interface into the interface a client expects.

RECOGNITION RADAR

CORE SIGNAL Two components are semantically compatible but their contracts do not match.

- A legacy or third-party API has the capability you need but exposes the wrong method/data shape.
- You cannot or should not modify either the client interface or the adaptee.
- Translation belongs at one boundary instead of leaking conversion logic across callers.

WHY THIS IS THE BEST FIT

- It preserves the client contract and quarantines vendor/legacy details in one small class.
- The rest of the system can test against a stable domain-facing interface.

TRADE-OFFS AND GUARDRAILS



COST

Another indirection layer; lossy semantic mapping can hide important mismatch.



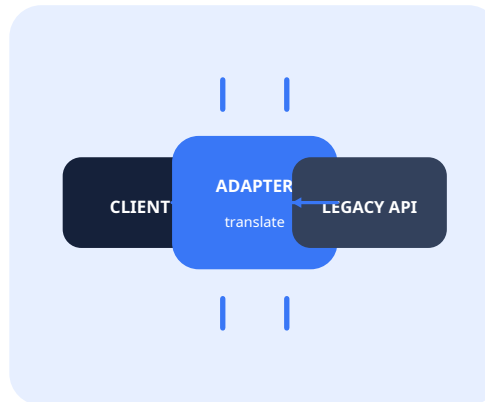
AVOID WHEN

When the contracts mean fundamentally different things; do not disguise incompatible semantics.

INTERVIEW-READY SENTENCE

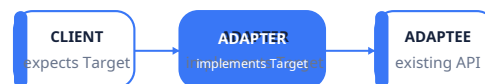
“ Use Adapter at integration boundaries when capability matches but the interface shape does not.

MEMORY ANCHOR



A travel plug changes the connection shape, not the electricity-consuming appliance.

STRUCTURE SNAPSHOT



FINGERPRINT

Interface mismatch

Compile-time wrapper

CONFUSION KILLER

Adapter changes an interface. Decorator keeps the same interface and adds behavior.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: PaymentPort is the target, StripeSdk the adaptee,
2 // StripeAdapter the translator.
3 // The client speaks the domain API; only the adapter knows vendor units
4 // and DTOs.
5
6 interface PaymentPort {
7     Receipt charge(Money amount);
8 }
9
10 // StripeSdk is an existing API with an incompatible
11 // method: StripeResult pay(long cents).
12
13 // Adapter exposes the interface our application expects.
14 final class StripeAdapter implements PaymentPort {
15     private final StripeSdk sdk;
16
17     StripeAdapter(StripeSdk sdk) {
18         this.sdk = sdk;
19     }
20
21     @Override
22     public Receipt charge(Money amount) {
23         // Translate our input into the adaptee's format.
24         StripeResult result = sdk.pay(amount.cents());
25
26         // Translate third-party output into our model.
27         return new Receipt(result.id(), result.ok());
28     }
29 }
30
31 // Client depends only on PaymentPort.
32 PaymentPort payments =
33     new StripeAdapter(new StripeSdk());
34 Receipt receipt = payments.charge(total);
35
36 // Vendor changes remain isolated inside this adapter.
37
38 // PATTERN CLUE: a useful class exists, but its interface does not match
39 // the client.
  
```

PRODUCTION NOTES

- Translate exceptions, units, nullability, and data models - not only method names.
- Keep vendor DTOs out of domain APIs.
- An anti-corruption layer is a larger architectural form of adapters.

Bridge

Separate an abstraction from its implementation so both axes can vary independently.

RECOGNITION RADAR

CORE SIGNAL Two independent dimensions are creating a subclass cross-product.

- You see classes such as RedCircle, BlueCircle, RedSquare, BlueSquare multiplying by combinations.
- A high-level abstraction and a low-level platform/vendor mechanism both need independent evolution.
- Composition can bind the two dimensions at runtime instead of inheritance fixing them together.

WHY THIS IS THE BEST FIT

- It replaces $N \times M$ subclasses with N abstractions plus M implementations.
- Each hierarchy has one reason to change and can be tested independently.

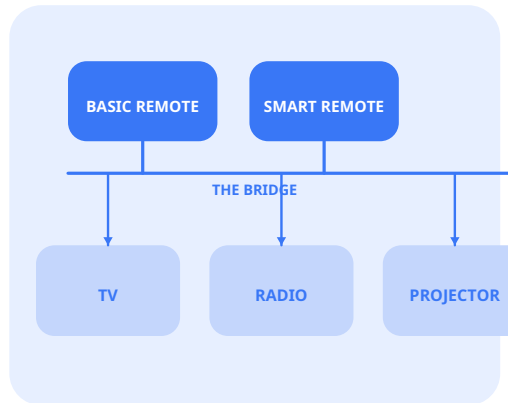
TRADE-OFFS AND GUARDRAILS

- ! COST**
More objects and up-front abstraction; overkill when one dimension is stable.
- ! AVOID WHEN**
When there is only one genuine variation axis or the implementation never changes.

INTERVIEW-READY SENTENCE

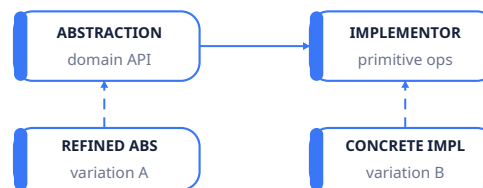
“ Use Bridge when two orthogonal dimensions vary and inheritance would create a combinatorial hierarchy.

MEMORY ANCHOR



One remote design can control many device types without multiplying every combination.

STRUCTURE SNAPSHOT



FINGERPRINT

Two variation axes Runtime composition Split hierarchies

CONFUSION KILLER

Strategy swaps an algorithm inside one context. Bridge models two long-lived class hierarchies.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: Shape is the abstraction axis; Renderer is the implementation
2 // axis.
3 // Composition connects the axes, allowing each hierarchy to evolve
4 // independently.
5
6 interface Renderer {
7     void drawCircle(int radius);
8 }
9
10 // One implementation axis: vector, raster, remote, etc.
11 final class VectorRenderer implements Renderer {
12     public void drawCircle(int r) { /* vector API */ }
13 }
14
15 // Abstraction stores an implementor instead of extending it.
16 abstract class Shape {
17     protected final Renderer renderer;
18     Shape(Renderer renderer) { this.renderer = renderer; }
19     abstract void draw();
20 }
21
22 final class Circle extends Shape {
23     private final int radius;
24
25     Circle(Renderer renderer, int radius) {
26         super(renderer);
27         this.radius = radius;
28     }
29
30     @Override
31     void draw() {
32         renderer.drawCircle(radius); // bridge delegation
33     }
34 }
35
36 // Shape type and rendering type now vary independently.
37 Shape icon = new Circle(new VectorRenderer(), 12);
38 icon.draw();
39
40 // PATTERN CLUE: two dimensions of variation are causing a subclass
41 // explosion.
  
```

PRODUCTION NOTES

- Name the implementation interface by capability, not with an Impl suffix.
- The abstraction may add domain policy while delegating primitive operations.
- Dependency injection naturally supplies the implementation side.

Composite

Treat individual objects and recursive groups through one uniform interface.

RECOGNITION RADAR

CORE SIGNAL The domain is a part-whole tree and clients should ignore leaf-vs-container differences.

- Objects naturally form a recursive hierarchy: UI tree, organization, expression, menu, filesystem.
- Operations should recurse through the structure without repeated instance of branching.
- A client should call the same operation on one item or a nested collection.

WHY THIS IS THE BEST FIT

- Uniform polymorphism makes recursive algorithms small and open to new leaf/composite types.
- Each composite owns traversal/aggregation of its children, preserving tree encapsulation.

TRADE-OFFS AND GUARDRAILS



COST

A common interface may become too broad; child-management rules and cycles need care.



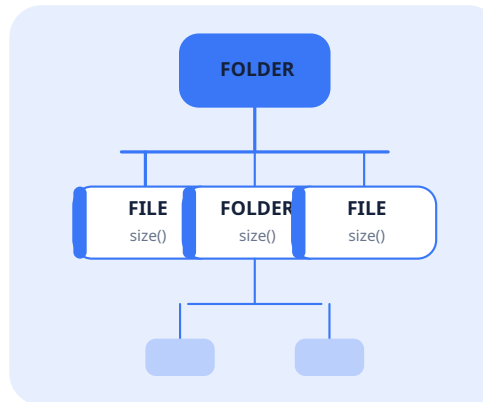
AVOID WHEN

When leaf and group semantics differ so much that a uniform contract becomes dishonest.

INTERVIEW-READY SENTENCE

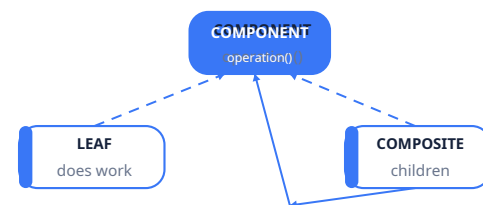
“ Use Composite when one and many must be treated uniformly inside a recursive part-whole structure.

MEMORY ANCHOR



A file and a folder both answer size(); folders simply aggregate children.

STRUCTURE SNAPSHOT



FINGERPRINT

Tree structure Uniform operation Recursive aggregation

CONFUSION KILLER

Composite models ownership/containment. Decorator usually wraps exactly one component to add behavior.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: Node is the component, FileNode a leaf, and Folder a
2 // composite.
3 // Leaves and groups both answer size(), so clients traverse the tree
4 // uniformly.
5
6 interface Node {
7     long size();
8 }
9
10 // Leaf has no children.
11 final class FileNode implements Node {
12     private final long bytes;
13     FileNode(long bytes) { this.bytes = bytes; }
14     public long size() { return bytes; }
15 }
16
17 // Composite can contain leaves or other composites.
18 final class Folder implements Node {
19     private final List<Node> children =
20         new ArrayList<>();
21
22     void add(Node child) {
23         children.add(child);
24     }
25
26     public long size() {
27         long total = 0;
28         for (Node child : children)
29             total += child.size(); // recursive call
30         return total;
31     }
32 }
33 Folder root = new Folder();
34 root.add(new FileNode(100));
35 Folder photos = new Folder();
36 photos.add(new FileNode(250));
37 root.add(photos);
38 long bytes = root.size(); // 350
39
40 // PATTERN CLUE: the domain is a part-whole tree and clients should ignore
41 // node kind.
```

PRODUCTION NOTES

- Prefer a narrow component interface containing truly common operations.
- Choose transparent child methods on Component or safer child methods only on Composite.
- Guard against cycles if the model is not guaranteed to be a strict tree.

Decorator

Add stackable responsibilities to an object while preserving its interface.

RECOGNITION RADAR

CORE SIGNAL Behavioral features must be composed per object and in different combinations.

- Subclasses are multiplying for combinations such as cached, logged, retried, encrypted, compressed.
- Each feature can wrap the same interface before/after delegating to another component.
- Capabilities need runtime ordering and stacking rather than one fixed inheritance chain.

WHY THIS IS THE BEST FIT

- Composition avoids subclass explosion and makes features independently reusable.
- The unchanged interface preserves substitutability for clients.

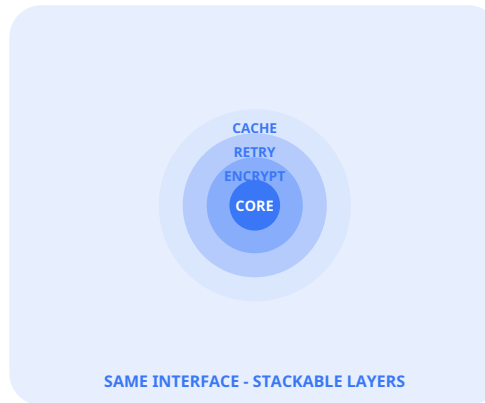
TRADE-OFFS AND GUARDRAILS

- COST**
Many small wrappers, order-sensitive behavior, and difficult debugging of deep chains.
- AVOID WHEN**
When one feature changes the public contract or requires coordination across unrelated methods.

INTERVIEW-READY SENTENCE

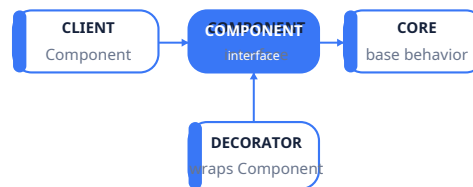
“ Use Decorator for optional, stackable per-object behavior that must keep the same contract.

MEMORY ANCHOR



Dress in layers: each layer adds warmth without becoming a different person.

STRUCTURE SNAPSHOT



FINGERPRINT

Same interface Wrap one component Stack at runtime

CONFUSION KILLER Proxy controls access to the wrapped object. Decorator enriches the object with responsibilities.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: DataSource is the component; every decorator stores another
2 // DataSource.
3 // Each wrapper adds behavior and delegates, so wrappers can be stacked at
4 // runtime.
5
6 interface DataSource {
7     byte[] read();
8 }
9
10 final class FileSource implements DataSource {
11     public byte[] read() { return loadBytes(); }
12 }
13
14 // Base decorator stores another component.
15 abstract class SourceDecorator implements DataSource {
16     protected final DataSource next;
17     SourceDecorator(DataSource next) { this.next = next; }
18 }
19
20 final class CachedSource extends SourceDecorator {
21     private byte[] cache;
22     CachedSource(DataSource next) { super(next); }
23
24     public byte[] read() {
25         if (cache == null) cache = next.read();
26         return cache;
27     }
28 }
29
30 final class DecryptingSource extends SourceDecorator {
31     DecryptingSource(DataSource next) { super(next); }
32     public byte[] read() { return decrypt(next.read()); }
33 }
34
35 // Wrappers compose from inside to outside.
36 DataSource source = new DecryptingSource(
37     new CachedSource(new FileSource()));
38 byte[] data = source.read();
39
40 // PATTERN CLUE: add optional responsibilities without multiplying
41 // subclasses.
  
```

PRODUCTION NOTES

- Define and test wrapper order explicitly: retry outside timeout differs from timeout outside retry.
- Forward identity-sensitive methods such as equals/hashCode carefully.
- Java I/O streams are the canonical example.

Facade

Expose a small task-oriented entry point over a complex subsystem.

RECOGNITION RADAR

CORE SIGNAL Most clients need a common workflow, not direct knowledge of subsystem choreography.

- Callers repeat the same multi-service sequence and know too many subsystem details.
- You want a stable use-case API while internals are refactored or replaced.
- The subsystem remains available for advanced cases, but common cases need a simple front door.

WHY THIS IS THE BEST FIT

- It reduces coupling and makes orchestration, ordering, and error translation consistent.
- The facade becomes a natural transaction/use-case boundary without hiding expert access.

TRADE-OFFS AND GUARDRAILS



COST

Can become a god service if it accumulates unrelated workflows or business rules.



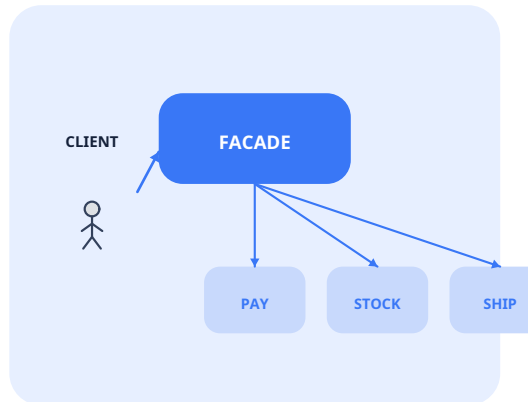
AVOID WHEN

When the facade only mirrors every subsystem method one-for-one and adds no useful simplification.

INTERVIEW-READY SENTENCE

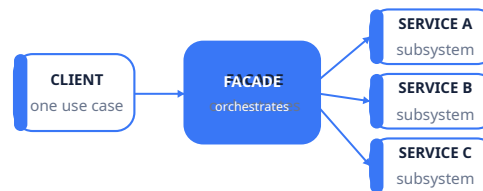
“ Use Facade to provide a cohesive use-case API over complex collaboration, not merely a forwarding class.

MEMORY ANCHOR



A hotel concierge coordinates rooms, transport, and dining through one request.

STRUCTURE SNAPSHOT



FINGERPRINT

Simplified API Subsystem boundary

CONFUSION KILLER Facade simplifies an interface for clients. Adapter converts one incompatible interface into another.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: CheckoutFacade coordinates Inventory, Payments, and Shipping.
2 // The client calls one business operation while the facade owns ordering
3 // and cleanup.
4
5 final class CheckoutFacade {
6     private final Inventory inventory;
7     private final Payments payments;
8     private final Shipping shipping;
9
10    CheckoutFacade(Inventory inv, Payments pay,
11                  Shipping ship) {
12        inventory = inv;
13        payments = pay;
14        shipping = ship;
15    }
16
17    // One use-case method hides subsystem sequencing.
18    OrderId place(Cart cart) {
19        inventory.reserve(cart.items());
20
21        try {
22            PaymentId payment =
23                payments.capture(cart.total());
24            return shipping.createOrder(cart, payment);
25        } catch (RuntimeException error) {
26            inventory.release(cart.items());
27            throw error;
28        }
29    }
30 }
31
32 // Client sees one small, task-oriented API.
33 CheckoutFacade checkout = new CheckoutFacade(
34     inventory, payments, shipping);
35 OrderId id = checkout.place(cart);
36
37 // Subsystem classes can still be used directly when needed.
38
39 // PATTERN CLUE: callers repeat a multi-service sequence and know too much.
  
```

PRODUCTION NOTES

- Expose business verbs such as placeOrder(), not subsystem-shaped getters.
- Keep multiple focused facades when a subsystem serves distinct use cases.
- Decide compensation/transaction semantics explicitly for partial failure.

Flyweight

Share immutable intrinsic state across many lightweight contextual objects.

RECOGNITION RADAR

CORE SIGNAL Huge object counts repeat the same immutable data and memory is a measured bottleneck.

- Profiling shows millions of similar objects dominated by duplicate strings, styles, glyphs, or metadata.
- State cleanly splits into shared intrinsic data and per-use extrinsic context.
- Shared objects can be immutable and obtained from a cache/factory by value key.

WHY THIS IS THE BEST FIT

- Deduplicating intrinsic state can reduce memory from $O(\text{objects} \times \text{shared-data})$ to $O(\text{objects} + \text{types})$.
- Immutability makes sharing safe and a factory enforces canonical instances.

TRADE-OFFS AND GUARDRAILS

! COST

More complex state separation, lookup overhead, and accidental sharing of mutable data.

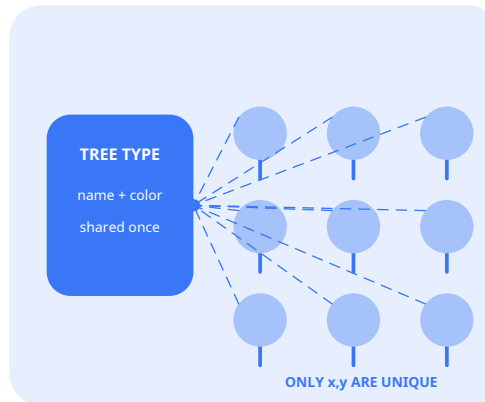
! AVOID WHEN

Before profiling, or when shared state is small relative to object overhead and complexity.

INTERVIEW-READY SENTENCE

“ Use Flyweight only after measuring duplicate intrinsic state at large scale.

MEMORY ANCHOR



A forest stores one tree-type palette while each tree carries only its location.

STRUCTURE SNAPSHOT



FINGERPRINT

Massive count Shared immutable state External context

CONFUSION KILLER

Singleton gives one instance total.
Flyweight gives one shared instance per intrinsic-state key.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: TreeType is shared intrinsic state; x and y are extrinsic
2 // per-tree state.
3 // The factory canonicalizes equal TreeType values so many objects reuse
4 // them.
5
6 // Intrinsic state: immutable and safe to share.
7 record TreeType(String name, Color color) {}
8
9 // Extrinsic state: unique position stays outside the type.
10 record Tree(int x, int y, TreeType type) {}
11
12 final class TreeTypeFactory {
13     private final Map<Key, TreeType> cache =
14         new HashMap<>();
15
16     TreeType get(String name, Color color) {
17         Key key = new Key(name, color);
18         return cache.computeIfAbsent(
19             key, k -> new TreeType(name, color));
20     }
21
22     private record Key(String name, Color color) {}
23 }
24
25 TreeType oak = factory.get("oak", GREEN);
26 Tree a = new Tree(10, 20, oak);
27 Tree b = new Tree(80, 40, oak);
28
29 // Many small context objects reuse one heavier value.
30 List<Tree> forest = new ArrayList<>();
31 for (Point p : points)
32     forest.add(new Tree(p.x(), p.y(), oak));
33
34 // Cache keys must include every intrinsic property.
35 boolean shared = (a.type() == b.type()); // true
36
37 // PATTERN CLUE: profiling shows many objects repeat large immutable data.
```

PRODUCTION NOTES

- Use value objects as cache keys; define equality and normalization carefully.
- Measure retained heap and hit rate before and after.
- String interning, glyph caches, and game-world object types are common forms.

Proxy

Stand in for another object to control access, lifecycle, or location.

RECOGNITION RADAR

CORE SIGNAL The client should see the same contract while access behavior is inserted transparently.

- Access needs authorization, lazy initialization, caching, rate limiting, logging, or remote transport.
- The real object is expensive, sensitive, unavailable locally, or should be created on first use.
- Callers should not change when the access policy changes.

WHY THIS IS THE BEST FIT

- The same interface preserves client code while centralizing access policy.
- The proxy controls when and whether the real subject is invoked.

TRADE-OFFS AND GUARDRAILS



COST

Hidden latency/failure, surprising identity semantics, and duplicated interface forwarding.



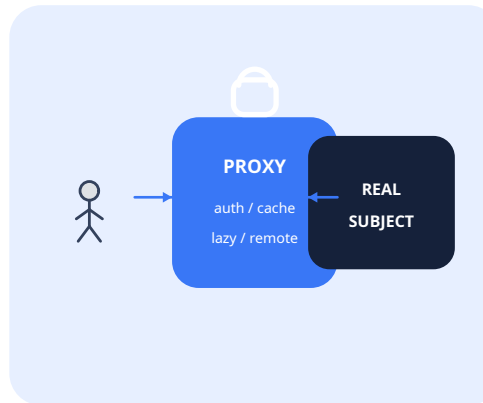
AVOID WHEN

When transparency would conceal important remote or expensive behavior from the API contract.

INTERVIEW-READY SENTENCE

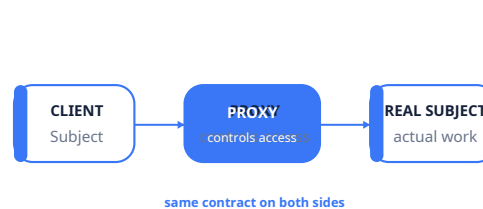
“ Use Proxy when the object contract stays the same but reaching the real object needs control.

MEMORY ANCHOR



A guarded gate looks like the entrance but checks permission before reaching the building.

STRUCTURE SNAPSHOT



FINGERPRINT

Same interface Access control Stand-in subject

CONFUSION KILLER

Decorator adds responsibilities as the main purpose. Proxy governs access to a real subject.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: SecuredStore and DatabaseStore share one interface; the proxy
2 // controls access.
3 // The client sees the same contract while the proxy checks, delays,
4 // caches, or logs.
5
6 interface DocumentStore {
7     Document get(Id id);
8 }
9
10 final class DatabaseStore implements DocumentStore {
11     public Document get(Id id) {
12         return database.load(id); // expensive real work
13     }
14 }
15
16 // Proxy has the same contract as the real subject.
17 final class SecuredStore implements DocumentStore {
18     private final DocumentStore real;
19     private final User user;
20
21     SecuredStore(DocumentStore real, User user) {
22         this.real = real;
23         this.user = user;
24     }
25
26     @Override
27     public Document get(Id id) {
28         // Control access before delegation.
29         if (!user.canRead(id)) throw new Forbidden();
30         Document document = real.get(id);
31         audit(user, id); // optional extra control
32         return document;
33     }
34 }
35
36 DocumentStore store = new SecuredStore(
37     new DatabaseStore(), currentUser);
38 Document document = store.get(id);
39
40 // PATTERN CLUE: access to the real object needs control without changing
41 // clients.
```

PRODUCTION NOTES

- Make remote boundaries explicit in naming or return types when latency/failure matters.
- Virtual proxies need safe lazy initialization under concurrency.
- Dynamic proxies/interceptors are infrastructure mechanisms; the design intent is still Proxy.

BEHAVIORAL

Control how objects decide and collaborate.

Where should changing behavior or communication policy live?

CATEGORY RULE

Replace conditionals and dependency webs with explicit objects for algorithms, states, requests, events, or traversal.

SELECTION LENS

PATTERN	DECISION VERB	DECISIVE RECOGNITION HACK	BINDING SHAPE
Chain of Responsibility	Pipeline	Ordered handlers independently act/pass	runtime chain
Command	Request object	Queue, log, replay, schedule, or undo action	deferred action
Interpreter	Grammar	Evaluate a tiny language as an AST	expression tree
Iterator	Traversal	Hide aggregate representation and cursor state	external cursor
Mediator	Coordination	Peers form a many-to-many dependency web	central policy
Memento	Snapshot	Restore private state without exposing it	opaque history
Observer	Notification	Dynamic one-to-many fan-out	subscribe/publish
State	Lifecycle	Internal mode changes legal behavior	internal transition
Strategy	Algorithm	Caller selects interchangeable approach	runtime delegation
Template Method	Skeleton	Fixed sequence with controlled subclass steps	inheritance hooks
Visitor	Operation	Stable types, frequently added cross-cutting work	double dispatch

REMEMBER THE DOMINANT NOUN

INTERACTION

Where should changing behavior or communication policy live?

COMMON CATEGORY MISTAKES

- Choosing by identical UML instead of intent
- Hiding event ordering/failure semantics
- Turning Mediator into a god object

BEST INTERVIEW MOVE

Compare your choice with the closest look-alike and explain the intent difference.

Chain of Responsibility

Pass a request through an ordered chain until handlers process or forward it.

RECOGNITION RADAR

CORE SIGNAL A request may need one or several independently configurable processing steps.

- The sender should not know which handler will act or how many handlers exist.
- Checks, filters, middleware, approval levels, or event processors form an ordered pipeline.
- Handlers need to be inserted, removed, or reordered without changing the requester.

WHY THIS IS THE BEST FIT

- It decouples sender from receiver and gives each handler one focused responsibility.
- Runtime chain composition makes processing policy configurable.

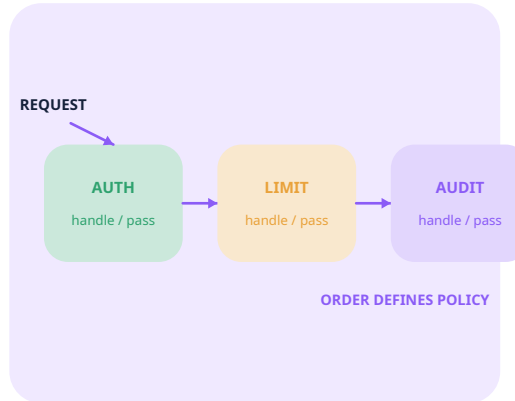
TRADE-OFFS AND GUARDRAILS

- ⚠ **COST**
Requests may go unhandled; order and stop/continue semantics can become hard to trace.
- ⚠ **AVOID WHEN**
When exactly one known service owns the request or strict transaction-wide coordination dominates.

INTERVIEW-READY SENTENCE

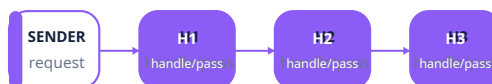
“ Use CoR when ordered handlers should independently decide to handle, enrich, reject, or forward a request.

MEMORY ANCHOR



Airport checkpoints: each station handles one concern, then sends you onward.

STRUCTURE SNAPSHOT



FINGERPRINT

Ordered pipeline Runtime chain Handle or forward

CONFUSION KILLER Decorator always wraps one component to enrich its operation. CoR may stop, skip, or pass to another handler.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: each Handler either handles the request or forwards it to
2 // next.
3 // Chain assembly determines order; individual handlers remain small and
4 // independent.
5
6 abstract class Handler {
7     private Handler next;
8     Handler linkWith(Handler next) {
9         this.next = next;
10        return next; // enables chain building
11    }
12
13    public final Response handle(Request request) {
14        if (canHandle(request))
15            return doHandle(request);
16        if (next != null)
17            return next.handle(request); // forward
18
19        throw new UnsupportedOperationException();
20    }
21
22    protected abstract boolean canHandle(Request r);
23    protected abstract Response doHandle(Request r);
24 }
25
26 final class AuthHandler extends Handler {
27     protected boolean canHandle(Request r) {
28         return r.type() == AUTH;
29     }
30     protected Response doHandle(Request r) {
31         return authenticate(r);
32     }
33 }
34
35 Handler root = new AuthHandler();
36 root.linkWith(new BillingHandler())
37     .linkWith(new FallbackHandler());
38 Response result = root.handle(request);
39
40 // PATTERN CLUE: several possible receivers may process a request at
41 // runtime.
  
```

PRODUCTION NOTES

- Define the boolean/enum outcome clearly: continue, handled, reject, or retry.
- Keep chain assembly in configuration/composition code.
- Servlet filters and HTTP middleware are common variants.

Command

Encapsulate a request as an object with everything needed to execute it.

RECOGNITION RADAR

CORE SIGNAL The action itself needs identity, storage, scheduling, history, undo, or transport.

- The requester should be decoupled from the receiver that performs the operation.
- Operations must be queued, logged, retried, composed, audited, or replayed.
- Undo/redo requires capturing prior state or an inverse operation.

WHY THIS IS THE BEST FIT

- Turning behavior into data enables lifecycle operations that plain method calls cannot support.
- Invoker, command, and receiver can evolve independently.

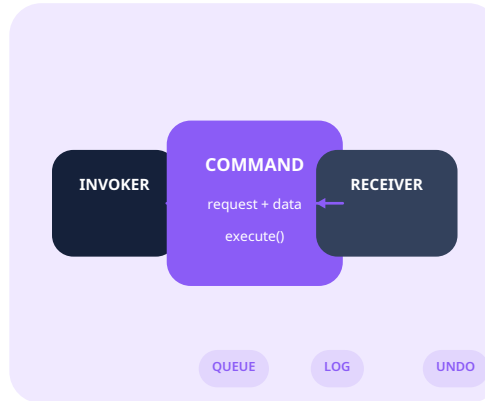
TRADE-OFFS AND GUARDRAILS

- ⚠ **COST**
One command type per action, payload/versioning concerns, and non-trivial reliable undo.
- ⚠ **AVOID WHEN**
For a direct synchronous call with no need for decoupling or command lifecycle.

INTERVIEW-READY SENTENCE

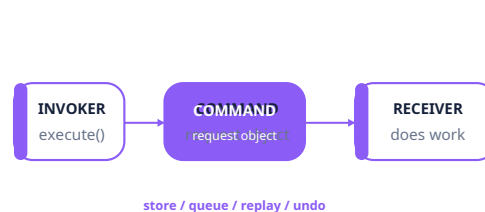
“ Use Command when a request must outlive, move beyond, or be managed separately from its caller.

MEMORY ANCHOR



A restaurant order ticket can be queued, logged, retried, or cancelled later.

STRUCTURE SNAPSHOT



FINGERPRINT

Request object Deferred execution Queue/log/undo

CONFUSION KILLER

Strategy represents interchangeable ways to do one job. Command represents a specific requested action.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: Command packages a request; TransferMoney stores receiver plus
2 // arguments.
3 // Because the action is an object, it can be queued, logged, retried, or
4 // undone.
5
6 interface Command {
7     void execute();
8     void undo();
9 }
10
11 // The command stores the receiver and all arguments.
12 final class TransferMoney implements Command {
13     private final Account from;
14     private final Account to;
15     private final Money amount;
16
17     TransferMoney(Account from, Account to, Money amount) {
18         this.from = from;
19         this.to = to;
20         this.amount = amount;
21     }
22
23     public void execute() {
24         from.debit(amount);
25         to.credit(amount);
26     }
27
28     public void undo() {
29         to.debit(amount);
30         from.credit(amount);
31     }
32 }
33
34 Command command = new TransferMoney(a, b, amount);
35 queue.add(command); // schedule, log, or retry
36 command.execute();
37 history.push(command); // later: history.pop().undo()
38
39 // PATTERN CLUE: operations need a lifecycle beyond an immediate method
40 // call.
  
```

PRODUCTION NOTES

- Commands should capture immutable identifiers/inputs, not fragile live UI objects.
- For distributed commands, design idempotency, deduplication, and versioned payloads.
- Undo may store a Memento or execute a compensating command.

Interpreter

Represent a small grammar as objects that evaluate expressions.

RECOGNITION RADAR

CORE SIGNAL A tiny, stable language or rule notation needs direct object-model evaluation.

- The domain has a compact grammar: filters, permissions, search rules, formulas, or expressions.
- Sentences can be represented as an abstract syntax tree of terminal and non-terminal expressions.
- Grammar complexity is small enough that one class per rule remains understandable.

WHY THIS IS THE BEST FIT

- The class structure mirrors grammar rules, making extension and recursive evaluation explicit.
- Parsed expressions can be composed, inspected, cached, and tested as objects.

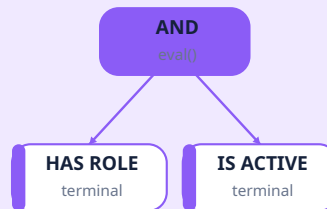
TRADE-OFFS AND GUARDRAILS

- ! COST**
Class explosion and poor performance/maintainability for complex languages.
- ! AVOID WHEN**
For serious languages; use parser generators, established expression engines, or AST visitors.

INTERVIEW-READY SENTENCE

“ Use Interpreter only for a small domain language whose grammar maps cleanly to an expression tree.

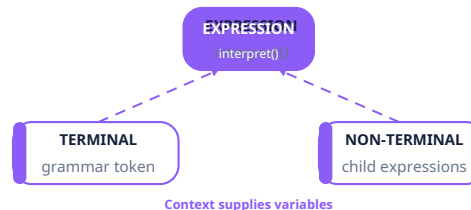
MEMORY ANCHOR



GRAMMAR BECOMES AN EXPRESSION TREE

A sentence becomes a tree; each grammar node knows how to interpret itself.

STRUCTURE SNAPSHOT



Context supplies variables

FINGERPRINT

Small grammar AST objects Recursive evaluate

CONFUSION KILLER Composite supplies the tree structure; Interpreter assigns grammar meaning and evaluation behavior to it.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: each Expr class represents one grammar rule; objects form an
2 // expression tree.
3 // eval() recursively interprets that tree against a Context.
4
5 interface Expr {
6     boolean eval(Context context);
7 }
8
9 // Terminal expression: one small grammar rule.
10 record HasRole(String role) implements Expr {
11     public boolean eval(Context c) {
12         return c.roles().contains(role);
13     }
14 }
15
16 record IsActive() implements Expr {
17     public boolean eval(Context c) {
18         return c.active();
19     }
20 }
21
22 // Non-terminal combines smaller expressions.
23 record And(Expr left, Expr right) implements Expr {
24     public boolean eval(Context c) {
25         return left.eval(c) && right.eval(c);
26     }
27 }
28
29 record Not(Expr inner) implements Expr {
30     public boolean eval(Context c) {
31         return !inner.eval(c);
32     }
33 }
34
35 // Build an object tree that represents the rule.
36 Expr rule = new And(
37     new HasRole("ADMIN"), new IsActive());
38 boolean allowed = rule.eval(userContext);
39
40 // PATTERN CLUE: a small, stable language can be modeled directly as
41 // objects.
  
```

PRODUCTION NOTES

- Separate parsing from evaluation; malformed input must not construct invalid trees.
- Use immutable expression nodes.
- Complex operations over an AST often migrate toward Visitor.

Iterator

Traverse a collection sequentially without exposing its representation.

RECOGNITION RADAR

CORE SIGNAL Traversal should be stable even when the underlying collection structure varies.

- Clients need sequential access but should not know indexes, tree links, buckets, or storage layout.
- The same aggregate needs multiple traversal orders or concurrent independent cursors.
- Traversal state belongs outside the collection rather than in global/client variables.

WHY THIS IS THE BEST FIT

- It separates traversal mechanics from aggregate structure and supports multiple traversals.
- A common iteration contract allows algorithms to work across collection types.

TRADE-OFFS AND GUARDRAILS



COST

Custom iterators add state and must define concurrent modification and exhaustion behavior.



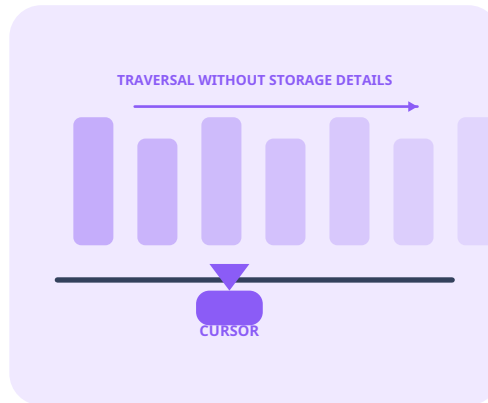
AVOID WHEN

When the language collection API already exposes the exact traversal semantics required.

INTERVIEW-READY SENTENCE

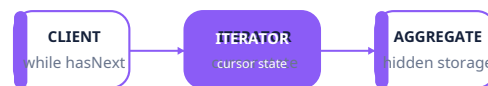
“ Use Iterator when traversal policy must be encapsulated independently from the aggregate representation.

MEMORY ANCHOR



A cursor moves through a library shelf while readers ignore how books are stored.

STRUCTURE SNAPSHOT



FINGERPRINT

Traversal cursor External state Representation hidden

CONFUSION KILLER

Iterator answers "how do I visit elements?"
Visitor answers "what new operation do I perform on each type?"

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: Playlist owns data; Iterator owns the traversal cursor and
2 // order.
3 // Clients ask hasNext()/next() without seeing indexes or internal storage.
4
5 final class Playlist implements Iterable<Song> {
6     private final List<Song> songs;
7
8     Playlist(List<Song> songs) {
9         this.songs = List.copyOf(songs);
10    }
11
12    @Override
13    public Iterator<Song> iterator() {
14        // Collection chooses its normal traversal.
15        return songs.iterator();
16    }
17
18    Iterator<Song> reverseIterator() {
19        return new Iterator<>() {
20            private int index = songs.size() - 1;
21
22            public boolean hasNext() {
23                return index >= 0;
24            }
25
26            public Song next() {
27                if (!hasNext())
28                    throw new NoSuchElementException();
29                return songs.get(index--);
30            }
31        };
32    }
33 }
34
35 // Client traverses without knowing internal storage.
36 for (Song song : playlist) play(song);
37 Iterator<Song> reverse = playlist.reverseIterator();
38
39 // PATTERN CLUE: expose traversal while preserving collection
40 // encapsulation.
```

PRODUCTION NOTES

- Prefer Java Iterable/Iterator unless the domain needs a distinct traversal contract.
- Document snapshot, fail-fast, weakly consistent, or live-view semantics.
- Lazy iterators can avoid materializing large result sets.

Mediator

Centralize interaction rules so peer objects do not reference each other directly.

RECOGNITION RADAR

CORE SIGNAL Many peers communicate in a dense web and interaction policy is scattered among them.

- Changing one component requires edits across many other peer components.
- UI controls, workflow participants, or modules need coordinated reactions and ordering.
- Peers should report events to one coordinator rather than hold pairwise dependencies.

WHY THIS IS THE BEST FIT

- It converts a many-to-many dependency web into many-to-one communication.
- Interaction policy becomes explicit, centralized, and independently testable.

TRADE-OFFS AND GUARDRAILS



COST

The mediator can become a god object and absorb domain logic that belongs elsewhere.



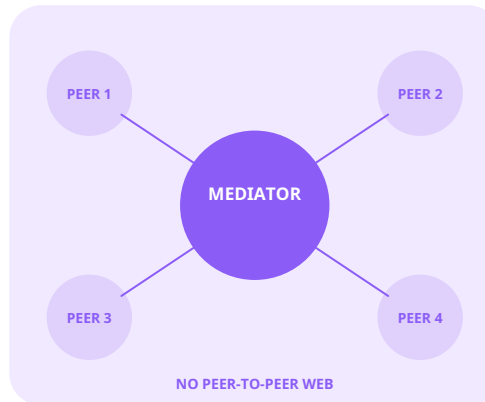
AVOID WHEN

When interactions are simple, stable, and naturally represented as direct collaboration.

INTERVIEW-READY SENTENCE

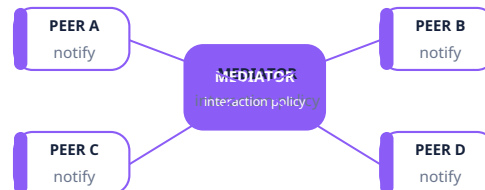
“ Use Mediator when the complexity lies in interactions among peers, not inside individual peers.

MEMORY ANCHOR



An air-traffic tower coordinates every aircraft; planes do not negotiate pairwise.

STRUCTURE SNAPSHOT



FINGERPRINT

Peer coordination Central policy Many-to-one links

CONFUSION KILLER

Observer broadcasts state changes. Mediator decides coordinated actions and often directs specific colleagues.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: controls are colleagues; SignupDialog is the central
2 // coordinator.
3 // Colleagues report events to the mediator instead of referencing one
4 // another.
5
6 interface DialogMediator {
7     void changed(Control source);
8 }
9
10 final class SignupDialog implements DialogMediator {
11     private final TextBox email;
12     private final CheckBox terms;
13     private final Button submit;
14
15     SignupDialog(TextBox email, CheckBox terms,
16                 Button submit) {
17         this.email = email;
18         this.terms = terms;
19         this.submit = submit;
20
21         // Controls report events only to the mediator.
22         email.onChange() -> changed(email);
23         terms.onChange() -> changed(terms);
24     }
25
26     @Override
27     public void changed(Control source) {
28         boolean ready =
29             email.isValid() && terms.isChecked();
30         submit.setEnabled(ready);
31     }
32 }
33
34 // Email, terms, and button never reference each other.
35 SignupDialog dialog =
36     new SignupDialog(email, terms, submit);
37
38 // PATTERN CLUE: peer objects have tangled many-to-many communication.
  
```

PRODUCTION NOTES

- Split one oversized mediator by bounded workflow or use case.
- Colleagues should expose capabilities; the mediator owns interaction sequencing.
- Event buses are not automatically mediators because they may carry no coordination policy.

Memento

Capture and restore an object snapshot without exposing its internals.

RECOGNITION RADAR

CORE SIGNAL State must be restored later, but encapsulation forbids external reconstruction of internals.

- Undo, checkpoints, rollback, drafts, or snapshots require returning to a prior state.
- Only the originator should understand the snapshot representation.
- A caretaker stores history but must not mutate or inspect internal state.

WHY THIS IS THE BEST FIT

- It preserves encapsulation while separating snapshot storage from snapshot creation/restoration.
- The originator can evolve internals while keeping the memento opaque.

TRADE-OFFS AND GUARDRAILS



COST

Memory growth, deep-copy cost, stale snapshot versions, and external-resource limits.



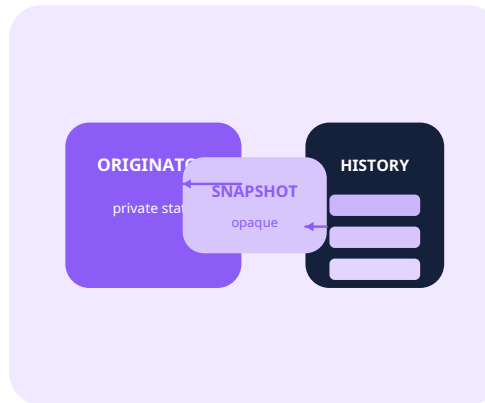
AVOID WHEN

For durable business history where events/audit records are needed, not opaque in-memory snapshots.

INTERVIEW-READY SENTENCE

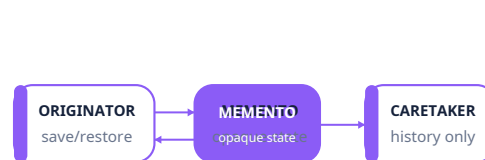
“ Use Memento when rollback requires private state and the snapshot must remain opaque to its caretaker.

MEMORY ANCHOR



A game save slot preserves the world state while the player sees only save/load.

STRUCTURE SNAPSHOT



FINGERPRINT

Opaque snapshot Restore later Originator owns format

CONFUSION KILLER

Command captures an action; Memento captures state. Undo commonly combines both.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: Editor is the originator, Snapshot the memento, history the
2 // caretaker.
3 // The caretaker stores opaque snapshots; only Editor knows how to restore
4 // them.
5
6 final class Editor {
7     // Immutable memento; caretaker cannot change it.
8     record Snapshot(String text, int cursor) {}
9
10    private String text = "";
11    private int cursor = 0;
12
13    Snapshot save() {
14        return new Snapshot(text, cursor);
15    }
16
17    void restore(Snapshot snapshot) {
18        text = snapshot.text();
19        cursor = snapshot.cursor();
20    }
21
22    void type(String value) {
23        text += value;
24        cursor = text.length();
25    }
26 }
27
28 // Caretaker stores snapshots without reading internals.
29 Deque<Editor.Snapshot> history =
30     new ArrayDeque<>();
31
32 history.push(editor.save());
33 editor.type("hello");
34
35 history.push(editor.save());
36 editor.type(" world");
37
38 editor.restore(history.pop()); // returns to "hello"
39
40 // PATTERN CLUE: undo needs prior state without exposing private fields.
```

PRODUCTION NOTES

- Use immutable snapshots and cap/compact history.
- For large state, store deltas or structural sharing instead of full copies.
- Do not confuse a Memento with persistence; durability and migration need explicit design.

Observer

Notify a dynamic set of subscribers when a subject changes or emits an event.

RECOGNITION RADAR

CORE SIGNAL One event must fan out to unknown or changing listeners without coupling the publisher to them.

- A subject should not know how many dependents exist or what each dependent does.
- Subscribers join/leave at runtime and react to state changes or domain events.
- UI binding, notifications, cache invalidation, or analytics need one-to-many propagation.

WHY THIS IS THE BEST FIT

- Publishers depend only on a listener contract, enabling open-ended fan-out.
- Subscriptions make the dependency dynamic and independently extensible.

TRADE-OFFS AND GUARDRAILS



COST

Implicit control flow, memory leaks, event storms, ordering ambiguity, and failure propagation.



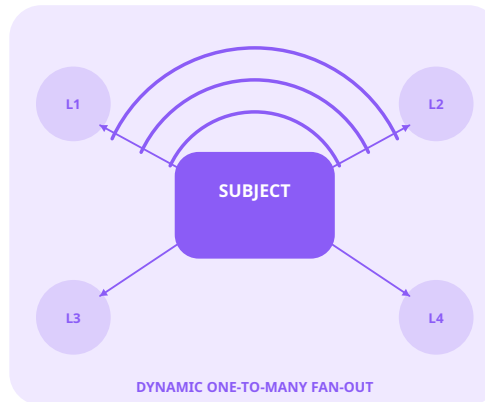
AVOID WHEN

When a synchronous direct call is required for correctness and the receiver is known.

INTERVIEW-READY SENTENCE

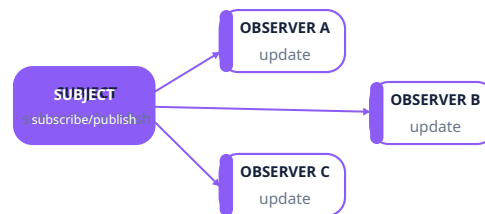
“ Use Observer for dynamic one-to-many notification; define delivery, ordering, and failure semantics explicitly. ”

MEMORY ANCHOR



One radio tower broadcasts; any tuned-in receiver reacts independently.

STRUCTURE SNAPSHOT



FINGERPRINT

One-to-many Runtime subscribe Push or pull event

CONFUSION KILLER

Observer distributes notifications without central decision logic. Mediator coordinates interactions among peers.

MINIMAL JAVA SHAPE

```
1 // ROLE MAP: PriceFeed is the subject; PriceListener implementations are
2 // observers.
3 // The subject publishes through an interface and does not know concrete
4 // reactions.
5
6 interface PriceListener {
7     void onPrice(Price price);
8 }
9
10 final class PriceFeed {
11     private final List<PriceListener> listeners =
12         new ArrayList<>();
13
14     void subscribe(PriceListener listener) {
15         listeners.add(listener);
16     }
17
18     void unsubscribe(PriceListener listener) {
19         listeners.remove(listener);
20     }
21
22     void publish(Price price) {
23         // Copy prevents mutation during notification.
24         for (PriceListener listener :
25             List.copyOf(listeners)) {
26             listener.onPrice(price);
27         }
28     }
29 }
30
31 // Subscribers can join or leave at runtime.
32 PriceListener screen = price -> display(price);
33 PriceListener alert = price -> notifyWhenLow(price);
34
35 feed.subscribe(screen);
36 feed.subscribe(alert);
37 feed.publish(new Price(99));
38
39 // PATTERN CLUE: one change must notify a dynamic set of dependents.
```

PRODUCTION NOTES

- Return a Subscription handle to make unsubscription hard to forget.
- Copy or use concurrent collections if listeners mutate subscriptions during callbacks.
- An in-process Observer is not automatically a durable message broker.

State

Let an object change behavior when its internal state changes.

RECOGNITION RADAR

CORE SIGNAL Behavior and legal transitions depend on a lifecycle mode, producing state-heavy conditionals.

- Methods contain repeated switch/if blocks over status across many operations.
- Each lifecycle state has distinct behavior and explicit legal transitions.
- The context should appear to change its class as state objects take over behavior.

WHY THIS IS THE BEST FIT

- It localizes state-specific behavior and transitions, eliminating scattered conditional logic.
- Adding a state is isolated and illegal operations can be rejected by that state.

TRADE-OFFS AND GUARDRAILS



COST

More classes; transitions can become distributed and difficult to visualize.



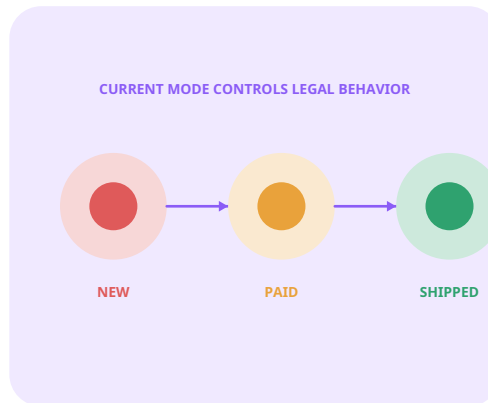
AVOID WHEN

When there are only two stable states with trivial behavior or state is pure data for reporting.

INTERVIEW-READY SENTENCE

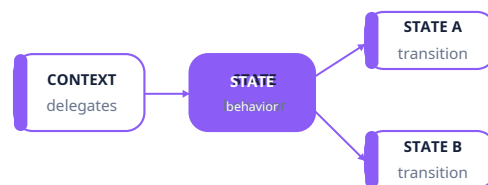
“ Use State when internal lifecycle mode controls behavior and transitions, not when a caller chooses an algorithm.

MEMORY ANCHOR



A traffic light is the same object, but each current color permits different transitions.

STRUCTURE SNAPSHOT



FINGERPRINT

Lifecycle modes Internal transition State-specific behavior

CONFUSION KILLER

Strategy is selected externally to choose an algorithm. State changes internally as the object evolves.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: Order is the context; each OrderState object owns one phase
2 // behavior.
3 // A state handles the event and may replace the context current state.
4
5 interface OrderState { void pay(Order o); void ship(Order o); }
6 // Each state owns valid actions and transitions.
7 final class NewState implements OrderState {
8     public void pay(Order order) {
9         order.changeTo(new PaidState());
10    }
11    public void ship(Order order) {
12        throw new IllegalStateException("pay first");
13    }
14 }
15 final class PaidState implements OrderState {
16     public void pay(Order order) {
17         throw new IllegalStateException("already paid");
18    }
19    public void ship(Order order) {
20        order.changeTo(new ShippedState()); // another state
21    }
22 }
23
24 final class Order {
25     private OrderState state = new NewState();
26
27     void pay() { state.pay(this); }
28     void ship() { state.ship(this); }
29
30     void changeTo(OrderState next) {
31         state = next; // explicit transition
32     }
33 }
34
35 Order order = new Order();
36 order.pay();
37 order.ship();
38
39 // PATTERN CLUE: behavior changes by mode and conditionals keep growing.
  
```

PRODUCTION NOTES

- Choose whether the Context or State owns transitions; keep one consistent source of truth.
- Singleton immutable state objects can be shared.
- A transition table/state machine library may be better for many states and events.

Strategy

Encapsulate interchangeable algorithms behind one contract.

RECOGNITION RADAR

CORE SIGNAL One task has several replaceable ways to perform it, chosen by caller or policy.

- A large conditional selects among algorithms such as pricing, routing, sorting, compression, or scoring.
- The algorithm should vary independently from the context that uses it.
- Selection may occur at runtime, through configuration, tests, or dependency injection.

WHY THIS IS THE BEST FIT

- It removes algorithm-selection conditionals and satisfies Open/Closed for new algorithms.
- Each strategy is independently testable and can be injected as a function/object.

TRADE-OFFS AND GUARDRAILS



COST

Clients must understand selection; tiny strategies can create excessive classes.



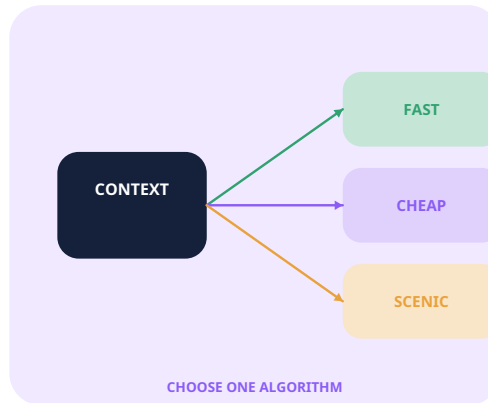
AVOID WHEN

When algorithms do not share a coherent contract or variation is unlikely and trivial.

INTERVIEW-READY SENTENCE

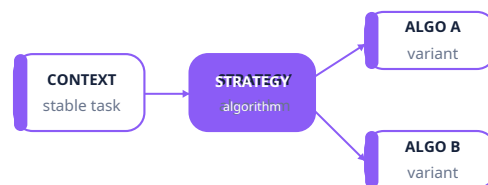
“ Use Strategy when the caller/policy selects one interchangeable algorithm for a stable task.

MEMORY ANCHOR



A navigation app selects driving, cycling, or walking while the trip goal stays the same.

STRUCTURE SNAPSHOT



FINGERPRINT

Algorithm family External selection One stable context

CONFUSION KILLER

State and Strategy share structure. Intent differs: lifecycle-driven internal change vs externally chosen algorithm.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: Checkout is the context; PricingStrategy is the replaceable
2 // algorithm.
3 // Selection happens outside the context; the context simply delegates
4 // price().
5
6 interface PricingStrategy { Money price(Cart cart); }
7 // Strategies should be stateless when possible.
8 final class RegularPricing implements PricingStrategy {
9     public Money price(Cart cart) {
10         return cart.subtotal();
11     }
12 }
13
14 final class FestivalPricing implements PricingStrategy {
15     public Money price(Cart cart) {
16         return cart.subtotal().multiply(0.90);
17     }
18 }
19 final class Checkout {
20     private PricingStrategy pricing;
21
22     Checkout(PricingStrategy pricing) {
23         this.pricing = pricing;
24     }
25
26     void use(PricingStrategy next) {
27         pricing = next; // switch algorithm at runtime
28     }
29
30     Money total(Cart cart) {
31         return pricing.price(cart);
32     }
33 }
34
35 // Caller chooses the algorithm; context only delegates.
36 Checkout checkout = new Checkout(new RegularPricing());
37 checkout.use(new FestivalPricing());
38 Money total = checkout.total(cart);
39
40 // PATTERN CLUE: several interchangeable ways perform the same
41 // responsibility.
  
```

PRODUCTION NOTES

- Use a lambda/function when the strategy is one stateless operation.
- Keep selection logic outside the Context, often in a factory or composition root.
- A strategy should own one algorithm, not become a general service locator.

Template Method

Fix an algorithm skeleton in a base class while subclasses customize selected steps.

RECOGNITION RADAR

CORE SIGNAL Several workflows share an invariant sequence but differ in a few bounded steps.

- Subclasses duplicate the same orchestration with small variations in selected operations.
- The base class must enforce ordering or invariants around extension points.
- Variation is naturally inheritance-based and known at compile time.

WHY THIS IS THE BEST FIT

- It centralizes the invariant algorithm and prevents subclasses from reordering critical steps.
- Primitive operations/hooks expose only the intended variation points.

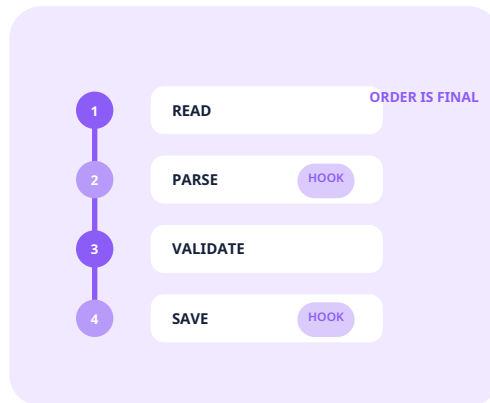
TRADE-OFFS AND GUARDRAILS

- COST**
Inheritance coupling, fragile base classes, and hook proliferation.
- AVOID WHEN**
When variation must change per object/runtime; prefer Strategy or composed step objects.

INTERVIEW-READY SENTENCE

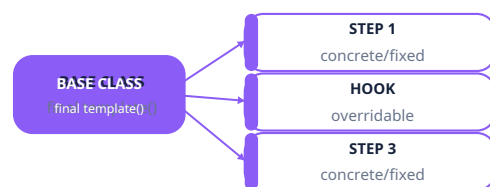
“ Use Template Method when the sequence is invariant and subclasses may customize only controlled steps.

MEMORY ANCHOR



A recipe fixes the sequence; chefs may customize sauce and garnish, not the food-safety order.

STRUCTURE SNAPSHOT



FINGERPRINT

Fixed skeleton Inheritance hooks Compile-time variation

CONFUSION KILLER Template Method varies steps through inheritance. Strategy delegates the whole varying algorithm by composition.

MINIMAL JAVA SHAPE

```

1 // ROLE MAP: DataImporter fixes the algorithm skeleton; subclasses vary
2 // selected steps.
3 // The final template method preserves order while hooks provide controlled
4 // variation.
5
6 abstract class DataImporter {
7     // Final keeps the algorithm order fixed.
8     public final void importData(Path path) {
9         byte[] raw = read(path); // stable step
10        List<Row> rows = parse(raw); // variable step
11        validate(rows); // optional hook
12        save(rows); // variable step
13    }
14
15    private byte[] read(Path path) {
16        return Files.readAllBytes(path); // error handling omitted
17    }
18
19    protected abstract List<Row> parse(byte[] raw);
20
21    protected void validate(List<Row> rows) {
22        // Default hook: subclasses may override.
23    }
24
25    protected abstract void save(List<Row> rows);
26 }
27
28 final class CsvImporter extends DataImporter {
29     protected List<Row> parse(byte[] raw) {
30         return csvParser.parse(raw);
31     }
32
33     protected void save(List<Row> rows) {
34         repository.saveAll(rows);
35     }
36 }
37
38 new CsvImporter().importData(file);
39
40 // PATTERN CLUE: related workflows share a sequence but differ in a few
41 // steps.
  
```

PRODUCTION NOTES

- Mark the template final to preserve sequence invariants.
- Keep hooks few, narrow, and documented as required or optional.
- Prefer composition when subclassing is used only to inject behavior.

Visitor

Add operations across a stable object structure without modifying element classes.

RECOGNITION RADAR

CORE SIGNAL Element types are stable, but new cross-cutting operations are added frequently.

- A heterogeneous object structure needs export, validation, pricing, metrics, or serialization operations.
- Adding each operation to every element would pollute domain classes or create repeated instances of logic.
- Double dispatch is acceptable: `element.accept(visitor)` selects the correct visit overload.

WHY THIS IS THE BEST FIT

- One operation stays together in one visitor while element classes remain focused.
- The compiler forces a visitor to handle all known element types.

TRADE-OFFS AND GUARDRAILS



COST

Adding a new element type changes every visitor; visitors may need privileged access to element data.



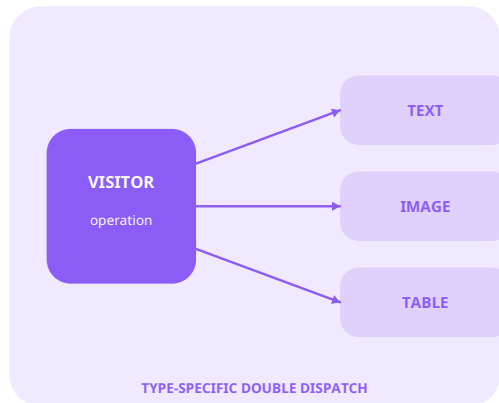
AVOID WHEN

When element types change often or only one/two natural operations exist.

INTERVIEW-READY SENTENCE

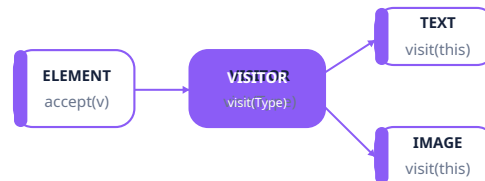
“ Use Visitor when operations grow faster than the stable set of element types.

MEMORY ANCHOR



A specialist inspector visits each machine type and performs type-specific work.

STRUCTURE SNAPSHOT



FINGERPRINT

Stable element set Growing operations Double dispatch

CONFUSION KILLER

Iterator controls traversal order. Visitor performs type-specific operations; the two often collaborate.



MINIMAL JAVA SHAPE

```
1 // ROLE MAP: elements accept a Visitor; overloaded visit() methods hold
2 // type-specific logic.
3 // Double dispatch selects both the runtime element type and the visitor
4 // operation.
5
6 interface Node {
7     void accept(Visitor visitor);
8 }
9
10 interface Visitor {
11     void visit(TextNode node);
12     void visit(ImageNode node);
13 }
14
15 final class TextNode implements Node {
16     private final String text;
17     TextNode(String text) { this.text = text; }
18     String text() { return text; }
19
20     public void accept(Visitor visitor) {
21         visitor.visit(this); // selects TextNode overload
22     }
23 }
24
25 final class ImageNode implements Node {
26     public void accept(Visitor visitor) {
27         visitor.visit(this); // selects ImageNode overload
28     }
29 }
30 final class HtmlVisitor implements Visitor {
31     public void visit(TextNode n) {
32         emit("<p>" + n.text() + "</p>");
33     }
34     public void visit(ImageNode n) { emit("<img>"); }
35 }
36
37 Visitor html = new HtmlVisitor();
38 for (Node node : document) node.accept(html);
39
40 // PATTERN CLUE: element types are stable but new cross-cutting operations
41 // arrive often.
```

PRODUCTION NOTES

- Traversal may live in Composite elements, a separate Iterator, or the visitor - choose deliberately.
- A sealed hierarchy plus pattern matching can be a simpler modern alternative.
- Keep visitors operation-focused; avoid turning one visitor into a grab bag.

Confusion clinic

When two patterns share a similar class diagram, decide by intent, owner of change, and lifecycle.

Factory Method / Abstract Factory / Builder

one product choice / compatible product family / stepwise configuration

Ask: Is the volatile decision type, family, or construction process?

Decorator vs Proxy

Both wrap the same interface. Decorator adds stackable responsibility; Proxy governs access to a real subject.

Ask: Is the purpose enrichment or control?

State vs Strategy

Same delegation shape. State changes internally with lifecycle; Strategy is selected externally by policy/caller.

Ask: Who initiates replacement - the object or its client?

Command vs Strategy

Command is a requested action with identity/lifecycle. Strategy is a reusable way to perform a task.

Ask: Must the object be queued, logged, transported, or undone?

Composite vs Decorator

Composite owns many children in a part-whole tree. Decorator wraps one component to add behavior.

Ask: Is the relationship aggregation or layering?

Adapter vs Facade

Adapter makes one existing contract usable. Facade creates a simpler task API over a subsystem.

Ask: Are we translating incompatibility or simplifying collaboration?

Bridge vs Strategy

Bridge separates two enduring variation hierarchies. Strategy swaps one algorithm inside one context.

Ask: Do two axes evolve independently, or only the algorithm?

Observer vs Mediator

Observer broadcasts to independent listeners. Mediator interprets peer events and coordinates directed actions.

Ask: Is there policy in the center, or only fan-out?

Iterator vs Visitor

Iterator encapsulates traversal state. Visitor encapsulates a new operation across heterogeneous element types.

Ask: Is variation in movement through objects or work performed on types?

Template Method vs Strategy

Template fixes sequence and varies steps through inheritance. Strategy delegates varying behavior by composition.

Ask: Must the base enforce order, or must behavior swap at runtime?

Patterns compose into systems

Real LLD solutions rarely use a single pattern. Combine intents, but keep each role narrow and justified.

Undoable editor

Command → Memento

WHY THE COMBINATION WORKS

Command gives each edit identity; Memento stores private snapshots for undo/redo.

GUARDRAIL

Keep a bounded history and choose inverse commands when snapshots are too large.

Cross-platform UI kit

Abstract Factory → Bridge

WHY THE COMBINATION WORKS

Factory selects a compatible widget family; Bridge separates widgets from platform rendering.

GUARDRAIL

Use one family factory at composition root and inject platform implementors.

Resilient API client

Facade → Adapter → Proxy → Decorator

WHY THE COMBINATION WORKS

Facade exposes use cases, Adapter translates vendor DTOs, Proxy controls remote access, Decorators add retry/cache/metrics.

GUARDRAIL

Make latency and failure visible; wrapper order must be tested.

Workflow engine

State → Command → CoR → Observer

WHY THE COMBINATION WORKS

State owns legal lifecycle behavior, Commands represent actions, CoR validates, Observer emits reactions.

GUARDRAIL

Do not let events become the source of truth unless event sourcing is intentional.

Document object model

Composite → Iterator → Visitor

WHY THE COMBINATION WORKS

Composite models the tree, Iterator controls traversal, Visitor adds export/validation operations.

GUARDRAIL

Best when node types are stable and operations grow.

Memory-efficient game scene

Composite → Flyweight → Prototype

WHY THE COMBINATION WORKS

Composite forms the scene graph, Flyweight shares asset metadata, Prototype clones configured entities.

GUARDRAIL

Define ownership and deep-copy boundaries explicitly.

LLD interview playbook

Model the domain first. Introduce a pattern only where a concrete variation or coupling pressure demands it.

A DISCIPLINED 7-STEP FLOW

1

Clarify scenarios

Actors, happy path, failure path, scale, concurrency, and likely change.

2

Extract nouns + verbs

Nouns suggest entities/value objects; verbs suggest responsibilities and use cases.

3

Protect invariants

Put state transitions and validation close to the object that owns them.

4

Mark variation points

Circle every switch/new/instanceof and every requirement phrased "may support more...".

5

Select by intent

Choose the smallest pattern that isolates that variation; compare its nearest alternative.

6

Draw relationships

Show owns/uses/delegates/observes. Avoid bidirectional links unless unavoidable.

7

Walk one scenario

Instantiate objects, call methods, transition state, and narrate error handling.

ANSWER TEMPLATE

"The volatile requirement is _____. Without an abstraction, _____ would depend on _____. I would use _____ because it moves that decision to _____. The stable contract is _____. The cost is _____, so I would avoid it when _____."

THE DIFFERENCE BETWEEN GOOD AND GREAT

A great answer names the alternative pattern and explains why its intent does not fit this force.

CODE REVIEW CHECKLIST

- ✓ Interfaces reflect capabilities, not pattern names.
- ✓ Dependencies enter through constructors.
- ✓ State/invariants have one owner.
- ✓ No switch merely moved into another switch.
- ✓ Tests can replace collaborators.

RED FLAGS

- "We should use a pattern" before requirements
- Pattern-named domain classes
- Unjustified Singleton/global state
- More indirection but no isolated change

Patterns through the SOLID lens

Patterns are recurring arrangements that often support SOLID. They do not automatically guarantee good design.



Single Responsibility

One reason to change per role.

Factory separates creation; Iterator separates traversal; Visitor separates an operation.



Open / Closed

Extend behavior without rewriting stable clients.

Strategy, Decorator, Observer, CoR, and Factory Method add variants behind contracts.



Liskov Substitution

Every implementation honors the semantic contract.

Decorator/Proxy must preserve Component behavior; State/Strategy variants must remain valid substitutes.



Interface Segregation

Clients depend only on capabilities they use.

Adapter exposes a narrow port; Bridge Implementor offers primitives; Composite needs a truthful common interface.



Dependency Inversion

High-level policy depends on abstractions.

Abstract Factory, Bridge, Strategy, Command, and Observer inject interfaces at stable boundaries.

PATTERN TEST: Does the design make the expected change local, explicit, substitutable, and testable?

One-page rapid recall

Trigger phrase -> pattern -> caution. Use this page for last-minute revision, then return to the detailed page for trade-offs.

CREATIONAL

Singleton

one process-wide identity

cost: global coupling

Factory Method

one product choice varies

cost: creator hierarchy

Abstract Factory

switch a matching family

cost: new product type is costly

Builder

many named construction options

cost: boilerplate

Prototype

copy configured exemplar

cost: deep-copy semantics

STRUCTURAL

Adapter

make incompatible API fit

cost: semantic mismatch

Bridge

split two variation axes

cost: premature dual hierarchy

Composite

treat one and many uniformly

cost: dishonest common API

Decorator

stack same-interface behavior

cost: order/deep chains

Facade

one simple subsystem entry

cost: god service

Flyweight

share repeated immutable state

cost: profile first

Proxy

control access to subject

cost: hidden latency/failure

BEHAVIORAL

Chain of Responsibility

ordered handle-or-forward pipeline

cost: order/unhandled request

Command

request needs lifecycle

cost: class/payload explosion

Interpreter

evaluate a tiny grammar

cost: grammar complexity

Iterator

hide traversal state

cost: concurrency semantics

Mediator

centralize peer coordination

cost: god coordinator

Memento

restore opaque snapshot

cost: snapshot memory

Observer

dynamic one-to-many fan-out

cost: leaks/order/storms

State

lifecycle mode controls behavior

cost: transition sprawl

Strategy

caller selects algorithm

cost: client selection burden

Template Method

fixed sequence, variable steps

cost: inheritance coupling

Visitor

new operations over stable types

cost: new element type costly

Use patterns as vocabulary, not as goals.

The best design is the simplest structure that protects today's invariants and makes the most likely change local tomorrow.

CANONICAL REFERENCES

- Gamma, Helm, Johnson, Vlissides - Design Patterns: Elements of Reusable Object-Oriented Software (1994).
- Freeman and Robson - Head First Design Patterns, 2nd edition.
- Joshua Bloch - Effective Java, 3rd edition.
- Robert C. Martin - Agile Software Development: Principles, Patterns, and Practices.

SCOPE NOTE

This cheatsheet covers the 23 Gang of Four object-oriented patterns. MVC, Dependency Injection, Repository, Unit of Work, Specification, CQRS, Event Sourcing, and architectural patterns are valuable, but they are not part of the canonical GoF 23.

RECOGNITION MANTRA

Find what varies. Keep what is stable. Move the decision behind a contract.